Project - Iteration I

System Modelling and Requirements Specification

Concordia University
SOEN 342 Software Requirements and Deployment

Fall 2026

Presented to Professor Ali Jannatpour

Paulina Aguayo Dupin - 40299470
Mia Matsumoto-Depatie - 40314818
Yana Vassilyev - 40270779

Oct 25th, 2026

Table of Content

1. Use Case Model **2**

2. UML Domain Model **3**

3. Fully-Dressed Use Case Scenarios **4**

3.1 UC1: Book Appointment 4

3.2 UC2: Register Patient Arrival 5

3.3 UC3: Conduct Consultation 5

4. System Sequence Diagrams **7**

4.1 UC1: Book Appointment (Success, Failure) 7

4.2 UC2: Register Patient Arrival (Success, Failure) 8

4.3 UC3: Conduct Consultation (Success, Failure 8

4.4 UC4: Create Referral (Success, Failure) 9

5. System Operations **10**

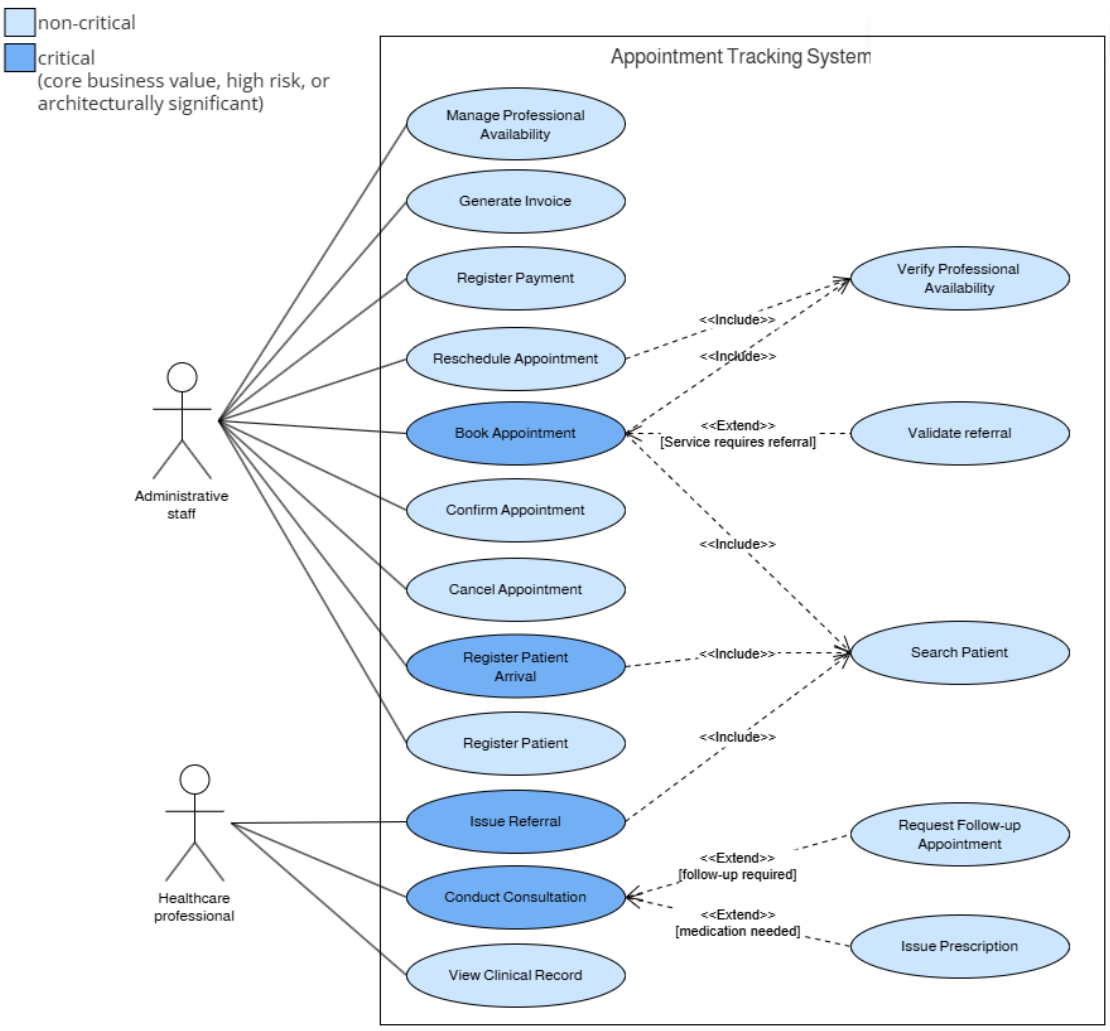
6. Operation Contracts **11**

UC1: Book Appointment 11

UC2: Register Patient Arrival 12

UC3: Conduct Consultation 12

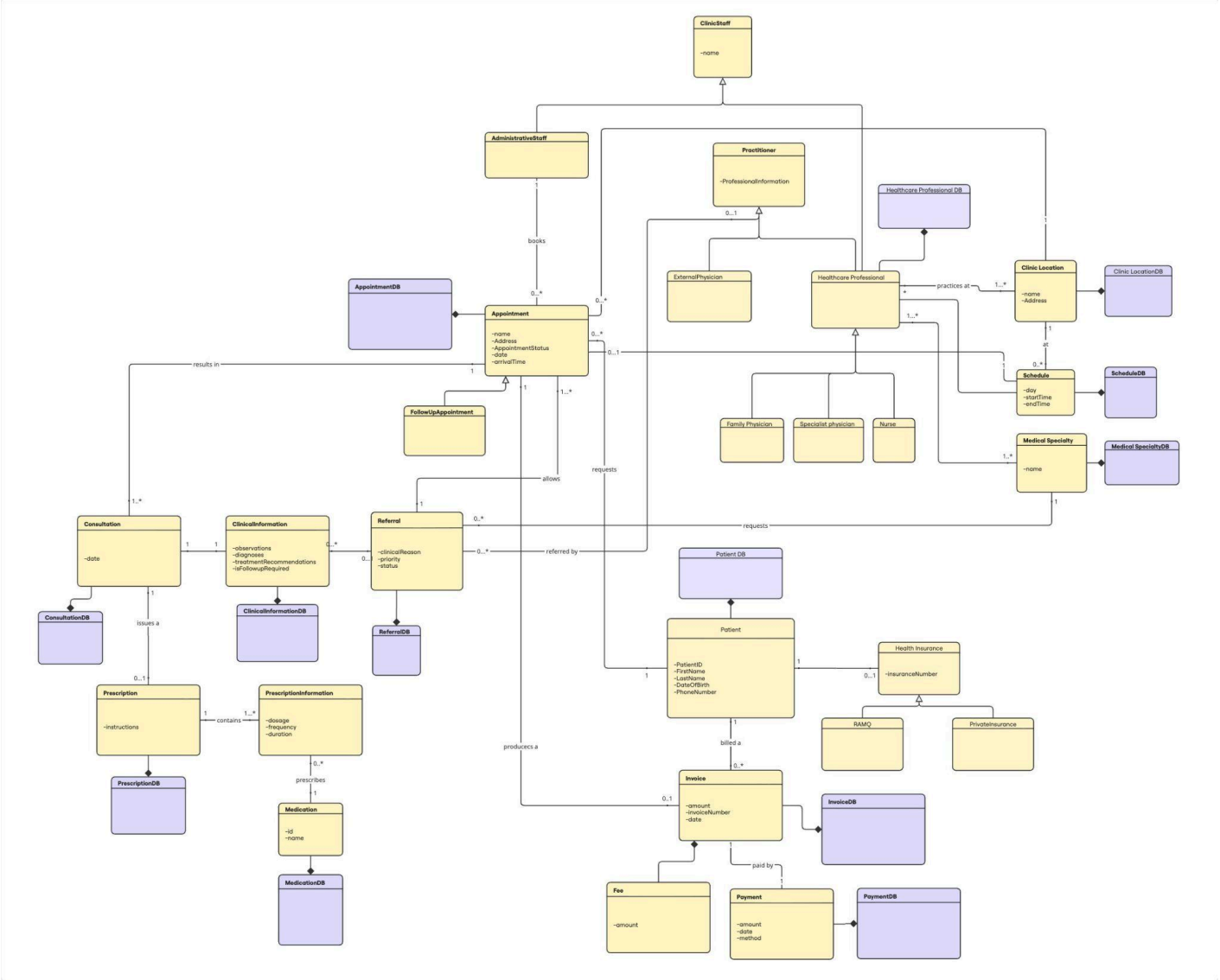
1. Use Case Model



Assumptions

A1. The system is a CLI application used only by clinic personnel. Patients do not interact with it directly. Their requests (e.g., appointments) are made in person or by phone and entered by administrative staff. Patients are therefore treated as stakeholders, not actors. This is consistent with the description, which defines user roles only for administrative staff and healthcare professionals.

2. UML Domain Model



3. Fully-Dressed Use Case Scenarios

3.1 UC1: Book Appointment

Primary Actor: Administrative staff
Stakeholders and Interests:

- Patient: Wants an appointment at a convenient time and location with a healthcare professional.
- Healthcare professional: Wants a schedule that respects their availability.
- Clinic: Wants no schedule conflicts and a record/history of the bookings.

Preconditions: The administrative staff is authenticated and the patient is registered in the system.
Success Guarantee (Postconditions): An appointment status is “Booked” for a patient, a service, a professional, a clinic location and a time slot.
Main success scenario (or basic flow):

1. Administrative staff enters the patient’s search criteria like their personal information.
2. System finds the matching patients.
3. Administrative staff starts a new appointment for the selected patient.
4. Administrative staff selects the requested service.
5. System displays the available professionals.
6. Administrative staff request for a time slot with a professional at a clinic location (includes Verify Professional Availability)
7. System confirms the slot is available
8. Administrative staff confirms the booking is complete
9. System saves the appointment and sends a booking confirmation with an appointment ID and changes status to “Booked”.

Extensions (or alternative flows):

- a*. If Administrative Staff cancels the booking at any time. No appointment is booked.
- 2a. If no patient matches the search criteria. Indicate error.
- 6a. If the service needs a referral. Staff attach the referral. The system confirms it is valid.
- 6b. If referral is invalid. Indicate error. Staff cancels and no appointment is booked.
- 8a. If no slot is available for the requested time. Indicate error. Staff requests another time or cancel the booking.
- 8b. If the patient already has an appointment at that time slot. Indicate error.

Special requirements: Command-line interface. Bookings are logged. Staff cannot see clinical information.
Technology and Data Variations List: Insurance number (RAMQ or private). Uninsured patients can be searched with other criteria.
Open issues: Can patients book directly or always through staff?

3.2 UC2: Register Patient Arrival

Primary Actor: Administrative staff
Stakeholders and Interests:

- Patient: Wants to check-in and have their appointment in time.
- Healthcare professional: Wants to know which patients have arrived.
- Clinic: Wants a record of who came to their appointment and when.

Preconditions: Administrative staff is authenticated. The patient has an appointment at the clinic location.
Success Guarantee (Postconditions): An appointment status is “Arrived” and arrival time is recorded. The patient is now on the waiting list of the healthcare professional. Check-in is recorded.
Main success scenario (or basic flow):

1. The patient arrives at the clinic. The administrative staff enters the patient’s search criteria.
2. The system finds the matching patients.
3. The administrative staff selects the correct patient.
4. The system shows the patient’s appointments for today.
5. The administrative staff logs the arrival of the patient for their appointment.
6. The system records the arrival time, sets the appointment to “Arrived” and adds the patient to the waiting list.

Extensions (or alternative flows):

- 2a. If no patient matches the search criteria. Indicate error.
- 4a. If the patient has no appointment today. Indicate error.
- 4b. If the appointment was cancelled. Indicate error.
- 5a. If the patient arrives too late (after the allowed grace period), log it. Staff can reschedule or continue if the professional agrees.

Special requirements: Command-line interface. Check-ins are logged. Staff cannot see clinical information.
Technology and Data Variations List: Insurance number (RAMQ or private). Uninsured patients can be searched with other criteria.
Open issues: Should the insurance be checked at check-in ? How long is the grace period ?

3.3 UC3: Conduct Consultation

Primary Actor: Healthcare professional
Stakeholders and Interests:

- Patient: Wants an accurate clinical record and confidentiality.
- Healthcare professional: Wants fast access to the patient’s history and a way to record the consultation.
- - Clinic: Wants to record every access to clinical data and the fees of the services.

Preconditions: Healthcare professional is authenticated. The patient has an appointment with this professional today.

Success Guarantee (Postconditions): The consultation is saved in the patient's record with the healthcare professional's notes. The appointment status is set to " Completed". Any follow-up request is also recorded and free is recorded for uninsured services.

Main success scenario (or basic flow):

1. A healthcare professional starts the consultation with the patient.
2. The system presents the patient's clinical information.
3. The healthcare professional records clinical notes.
4. The system confirms the notes recorded.
5. The healthcare professional confirms the completion of the consultation.
6. The system saves the consultation to the clinical record, sets the appointment to "Completed.

Extensions (or alternative flows):

- 1a. If the patient's arrival was not registered for this appointment. Indicate error.
- 1b. If the appointment is an assignment to another professional. Indicate error. Deny access to the record of the patient.
- 3a. If medication needs to be prescribed, the professional opens a new prescription.
- 3b. If a medication does not exist. Indicate error.
- 3c. If a follow-up is needed, the professional needs to record a follow-up request.
- 3d. If a specialist is needed, the professional can create a referral.
- 5a. If nothing was recorded during the consultation. Indicate error.
- 6a. If the service is uninsured, the system records it and sends an invoice.

Special requirements: Command-line interface. Only healthcare professionals can see clinical information.

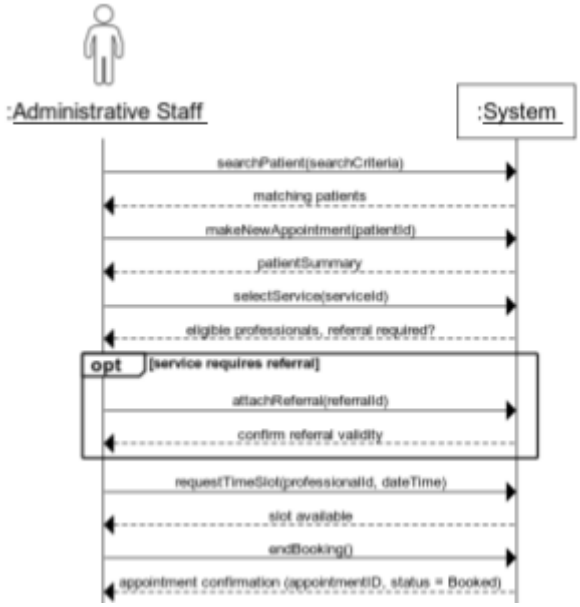
Technology and Data Variations List: Clinical notes are entered as text.

Open issues: Can a consultation be saved as a draft ?

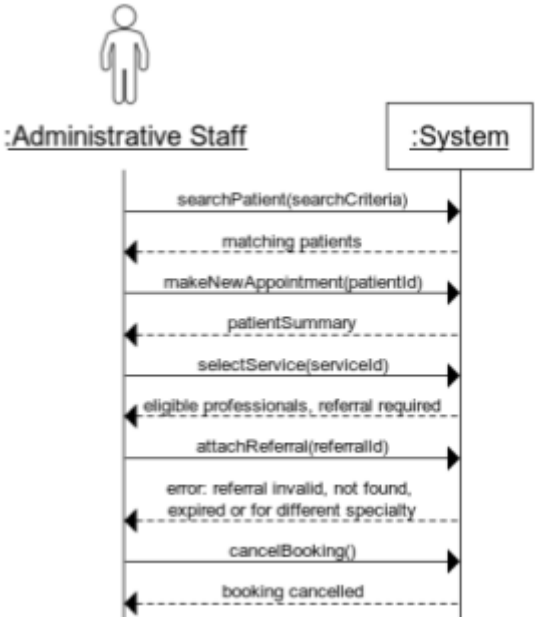
4. System Sequence Diagrams

4.1 UC1: Book Appointment (Success, Failure)

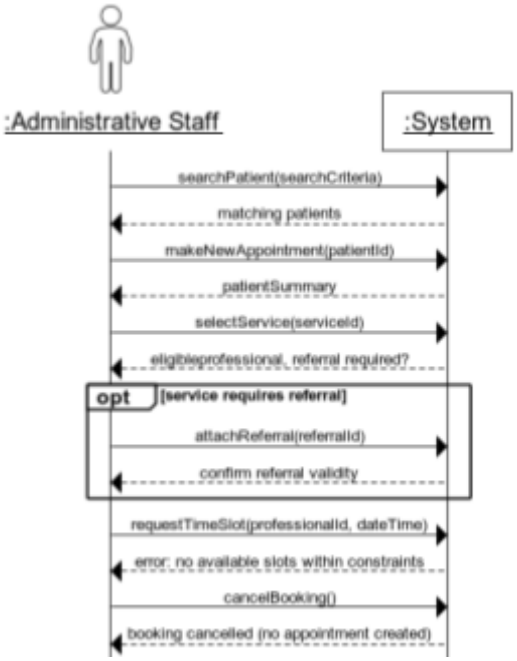
UC1: Book Appointment - Success



UC1: Book Appointment - Failure (required referral invalid)

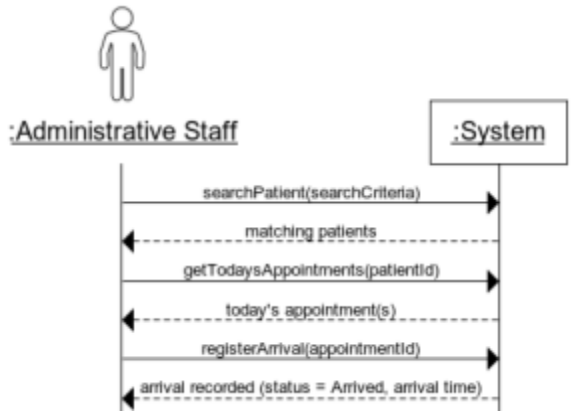


UC1: Book Appointment - Failure (no slots available)

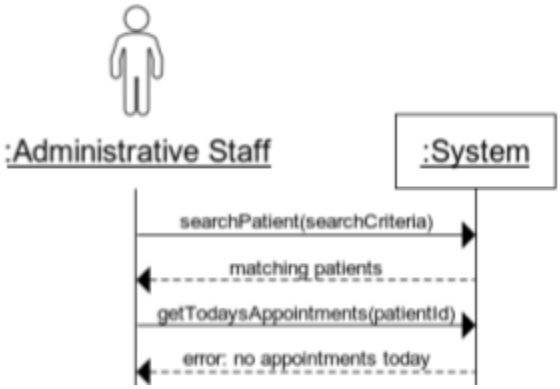


4.2 UC2: Register Patient Arrival (Success, Failure)

UC2: Register Patient Arrival - Success



UC2: Register Patient Arrival - Failure (no valid appointment)

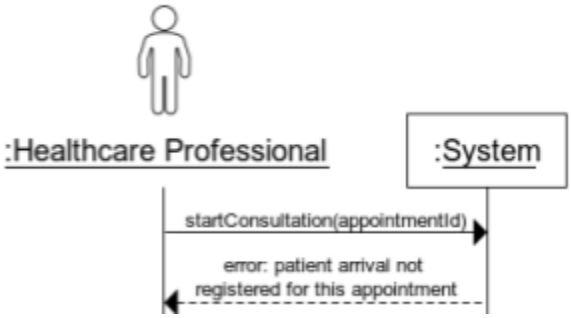


4.3 UC3: Conduct Consultation (Success, Failure

UC3: Conduct Consultation - Success

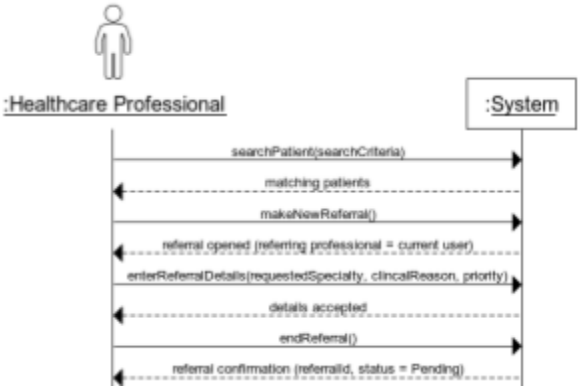


UC3: Conduct Consultation - Failure (arrival not registered)

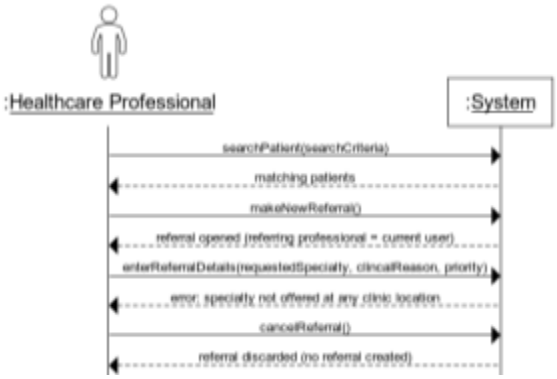


4.4 UC4: Create Referral (Success, Failure)

UC4: Issue Referral - Success



UC4: Issue Referral - Failure (specialty not offered)



5. System Operations

Each system operation below is a message an actor sends to the system, but the operations marked as query only get information and do not change the state of the system. They do not have operation contracts in the next section.

UC1: Book Appointment

- searchPatient (searchCriteria) [query]
- makeNewAppointment (patientId)
- selectService (serviceId)
- attachReferral (referralId)
- requestTimeSlot (professionalId, locationId, dateTime)
- endBooking ()
- cancelBooking ()

UC1: Register Patient Arrival

- searchPatient (searchCriteria) [query]
- getTodaysAppointments (patientId) [query]
- registerArrival (appointmentId)

UC3: Conduct Consultation

- startConsultation (appointmentId)
- recordClinicalNotes (observation, diagnoses, treatment)
- makeNewPrescription ()
- addMedication (medicationId, dosage, frequency, duration)
- requestFollowUp (timeFrame, reason)
- endConsultation ()

6. Operation Contracts

UC1: Book Appointment

Contract CO1: makeNewAppointment

- Operation: makeNewAppointment (patientId:String)
- Cross References: Use Case: Book Appointment
- Preconditions: The administrative staff member is logged in. A patient with patientId exists.
- Postconditions:
    - An appointment was created associated with a patient
    - Status is set to pending

Contract CO2: selectService

- Operation: selectService(serviceId: String)
- Cross References: Use Case: Book Appointment
- Preconditions: An appointment is being booked. A service with a serviceId exists.
- Postconditions:
    - The booking was associated with a service

Contract CO3: attachReferal

- Operation: attachReferral (referralId: String)
- Cross References: Use Case: Book Appointment
- Preconditions: An appointment is being booked and service requires referral.
- Postconditions:
    - A booking was associated with a referral

Contract CO4: requestTimeSlot

- Operation: requestTimeSlot(professionalId: String, locationId: String, dateTime: DateTime)
- Cross References: Use Case: Book Appointment
- Preconditions: Appointment is being booked and has a service. Healthcare professional offers this service at that location and timeslot is available. The patient has no other appointment at the same time.
- Postconditions:
    - The booking was associated with healthcare professional
    - The booking was associated with Location
    - The booking was associated with Timeslot
    - The booking was associated with dateTime

Contract CO5: endBooking

- Operation: endBooking()
- Cross References: Use Case: Book Appointment
- Preconditions: An appointment is being booked and has a patient, as service, a healthcare profession, a location and a timeslot.
- Postconditions:
    - Appointment status is set to “Booked”
    - Timeslot is not available anymore
    - LogEntry is created

Contract CO6: cancelBooking

- Operation: cancelBooking()
- Cross References: Use Case: Book Appointment
- Preconditions: An appointment is being booked.
- Postconditions:
    - All associations are deleted
    - Appointment is deleted

UC2: Register Patient Arrival

Contract CO7: registerArrival

- Operation: registerArrival (appointmentId:String)
- Cross References: Use Case: Register Patient Arrival
- Preconditions: The admin member is logged in. An appointment exists at the moment of the arrival.
- Postconditions:
    - Appointment is set to “Arrived” with timestamp
    - LogEntry created
    - Register arrival

UC3: Conduct Consultation

Contract CO8: startConsultation

- Operation: startConsultation (appointmentId:String)
- Cross References: Use Case: Conduct Consultation
- Preconditions: The healthcare profession is logged in. An appointment exists and has the status “Arrived”.
- Postconditions:
    - A consultation was created
    - Consultation startTime is set to current time
    - Consultation associated with appointment

- Consultation associated to patient's record
- LogEntry is created
- LogEntry is associated with Healthcare Professional

Contract CO9: recordClinicalNotes

- Operation: recordClinicalNotes(observations:String, diagnoses: String, treatment: String)
- Cross References: Use Case: Conduct Consultation
- Preconditions: A consultation is in progress.
- Postconditions:
    - Consultation has observation attribute set

Contract CO10: makeNewPrescription

- Operation: makeNewPrescription()
- Cross References: Use Case: Conduct Consultation
- Preconditions: A consultation is in progress.
- Postconditions:
    - Prescription was created
    - Set date attribute of prescription to today
    - Prescription associated to consultation

Contract CO11: addMedication

- Operation: addMedication(medicationId:String, dosage: String, frequency: String, duration: String)
- Cross References: Use Case: Conduct Consultation
- Preconditions: A medication with medicationId exists.
- Postconditions:
    - Prescription line instance is created
    - Prescription line associated to prescription
    - Prescription line associated with Medication

Contract CO12: requestFollowUp

- Operation: requestFollowUp (timeFrame: String, reason: String)
- Cross References: Use Case: Conduct Consultation
- Preconditions: A consultation is in progress.
- Postconditions:
    - FollowUpRequest is created
    - FollowUpRequest associated with Consultation

Contract CO13: endConsultation

- Operation: endConsultation()
- Cross References: Use Case: Conduct Consultation
- Preconditions: A consultation is in progress and notes have been recorded.
- Postconditions:
    - Consultation endTime is set to current time
    - Appointment is set to “Completed”
    - Log Entry is created
    - LogEntry action set to “Record consultation”
    - LogEntry associated to Healthcare Professional